



Exercise - Network Virtualization using Mininet

TSM - Advanced Communication Architecture

Dimitri Julmy

-

University of Applied Sciences and Arts of Western Switzerland (HES-SO)
Master of Science in Engineering (MSE)
Information and cyber security (ICS)

Repository URI	https://github.com/HezelTm/tsm_advcomarc
Creation date	2026-05-18
Submission date	2026-05-19

Introduction

Ce laboratoire explore les concepts fondamentaux du *Software-Defined Networking* (SDN) à travers la simulation d'un réseau virtuel avec *Mininet* et le contrôleur *POX*. Le SDN repose sur la séparation du plan de contrôle et du plan de données : le contrôleur décide des règles de transfert, et les *switches* les appliquent via le protocole *OpenFlow*.

L'environnement déployé repose sur une machine virtuelle Ubuntu 16.04 (*mininet-2.3.0*) importée sous *VirtualBox* et accessible en SSH depuis la machine hôte à l'adresse 192.168.56.101. Le contrôleur *POX* (*forwarding.l2_pairs*) est lancé sur la VM et écoute sur le port 6633. Les topologies simulées (*single,3,linear,3,linear,4*) sont instanciées via la CLI *Mininet*. Les captures réseau sont effectuées avec *tcpdump* directement sur la VM, puis transférées sur la machine hôte via *scp* pour analyse dans *Wireshark*.

L'objectif est d'observer et d'analyser les échanges réseau générés par *OpenFlow*, ARP, ICMP et HTTP, afin de comprendre comment le plan de contrôle SDN gère l'initialisation, l'apprentissage des adresses MAC et l'installation des règles de flux dans les *switches*.

Implementation

Mise en place de l'environnement de simulation

1. Referring to the Lab. Simulation reported at the end of OpenFlow course, please implement the simulations reported on slide 45, 46 and 47 of the course module on OpenFlow.

Premièrement, installer VirtualBox (procédure pour Arch OS) (<https://wiki.archlinux.org/title/VirtualBox>):

```
1 yay -S virtualbox
```

Puis télécharger l'image mininet-2.3.0-210211-ubuntu-16.04.7-server-amd64-ovf depuis la source officielle : <https://github.com/mininet/mininet/releases>.

Une fois l'image téléchargée, la décompresser et l'importer dans le dossier voulu.

```
1 unzip mininet-2.3.0-210211-ubuntu-16.04.7-server-amd64-ovf.zip -d ~/Isos/
```

- -d ~/Isos/ : spécifie le répertoire de destination de la décompression.

Pour créer la machine virtuelle, suivre le guide officiel : <https://mininet.org/vm-setup-notes/>.

1. Sur VirtualBox, cliquer sur Import et sélectionner le fichier mininet-2.3.0-210211-ubuntu-16.04.7-server-amd64-ovf. Valider l'importation.
2. Sélectionner la machine virtuelle importée, cliquer sur Settings en mode Expert puis Network. Sélectionner Host-only Adapter pour Adapter 1. Sélectionner vboxnet0 pour Name. Valider les changements.

Si vboxnet0 n'est pas disponible, il faut le créer à l'aide des commandes suivantes :

```
1 # Create a host-only network interface
2 sudo vboxmanage hostonlyif create
3 # Check if the interface was created successfully
4 vboxmanage list hostonlyifs
```

- hostonlyif create : crée une nouvelle interface réseau hôte-uniquement virtuelle (vboxnet0).
- list hostonlyifs : liste toutes les interfaces hôte-uniquement existantes pour confirmer la création.

Exécuter la machine virtuelle et se connecter avec les identifiants suivants :

- Username : mininet
- Password : mininet

Trouver l'adresse IP de la machine virtuelle à l'aide de la commande suivante (à exécuter dans la machine virtuelle). Pour ce travail, l'adresse IP est : 192.168.56.101.

```
1 # Get eth0 IP address
2 ifconfig eth0
```

- `eth0` : nom de l'interface réseau correspondant à l'adaptateur hôte-uniquement VirtualBox dans la VM.

Pour faciliter la connexion SSH par nom, ajouter une entrée dans le fichier `/etc/hosts` de la machine hôte :

```
1 echo "192.168.56.101 mininet-vm" | sudo tee -a /etc/hosts
```

- `tee` : lit depuis stdin et écrit simultanément dans un fichier et sur stdout.
- `-a` : mode *append* — ajoute à la fin du fichier sans écraser le contenu existant.

Se connecter en SSH à la machine virtuelle depuis la machine hôte :

```
1 ssh -Y mininet@mininet-vm
```

- `-Y` : active le transfert X11 de confiance (*trusted X11 forwarding*), permettant d'afficher des interfaces graphiques (comme `xterm` ou `Wireshark`) depuis la VM sur la machine hôte.

Démarrer le contrôleur POX dans un premier terminal SSH :

```
1 # Run POX controller
2 sudo ~/pox/pox.py forwarding.l2_pairs info.packet_dump samples.pretty_log log.level --DEBUG
```

- `forwarding.l2_pairs` : module d'apprentissage L2 réactif — apprend les paires MAC src/dst et installe des règles de flux dans les commutateurs.
- `info.packet_dump` : affiche le contenu des paquets reçus par le contrôleur (utile pour le débogage).
- `samples.pretty_log` : formate les logs de façon lisible avec horodatage et niveau de sévérité.
- `log.level --DEBUG` : fixe le niveau de verbosité des logs à `DEBUG` pour voir tous les événements OpenFlow.

Puis, démarrer la topologie souhaitée dans une autre session SSH.

- `single,3` : un commutateur central connecté à 3 hôtes (`h1`, `h2`, `h3`).

```
1 sudo mn -x --topo single,3 --controller remote
```

- `linear,3` : une chaîne linéaire de 3 commutateurs, chacun connecté à un hôte (`h1`, `h2`, `h3`).

```
1 sudo mn -x --topo linear,3 --controller remote
```

- `linear,4` : une chaîne linéaire de 4 commutateurs, chacun connecté à un hôte (`h1`, `h2`, `h3`, `h4`).

```
1 sudo mn --topo linear,4 --controller remote
```

Les paramètres communs à ces commandes `mn` :

- `-x` : ouvre un terminal `xterm` pour chaque nœud du réseau (requiert le transfert X11 activé par `ssh -Y`).
- `--topo <type>,<n>` : définit la topologie réseau à instancier (`single`, `linear`, `tree`, etc.) avec le nombre de nœuds.
- `--controller remote` : indique à Mininet d'utiliser un contrôleur distant plutôt que le contrôleur interne – se connecte à `localhost:6633` par défaut (port standard OpenFlow 1.0).

Démarrer le serveur HTTP sur `h1` et effectuer une requête depuis `h2` :

```
1 h1 python -m SimpleHTTPServer 80 &
2 h2 wget -O - h1
```

- `-m SimpleHTTPServer` : lance le module Python `SimpleHTTPServer` comme programme principal (sert le répertoire courant via HTTP).
- `80` : port d'écoute du serveur HTTP.
- `&` : exécute le serveur en arrière-plan pour libérer le terminal Mininet.
- `wget -O -` : télécharge la ressource et redirige le contenu vers `stdout` au lieu de l'enregistrer dans un fichier.

Vérification des composants

2. Please verify the functionality of all system components.

Une fois Mininet démarré, les commandes suivantes vérifient que tous les composants fonctionnent correctement. Les exemples ci-dessous sont basés sur la topologie `single`, 3. Toutes les commandes sont à exécuter dans la CLI Mininet.

Lister les nœuds et liens du réseau :

```
1 # List nodes
2 nodes
3 # Output
4 available nodes are:
5 c0 h1 h2 h3 s1
6
7 # List links
8 net
9 # Output
10 h1 h1-eth0:s1-eth1
11 h2 h2-eth0:s1-eth2
12 h3 h3-eth0:s1-eth3
13 s1 lo: s1-eth1:h1-eth0 s1-eth2:h2-eth0 s1-eth3:h3-eth0
14 c0
```

Vérifier la configuration réseau de h1 :

```
1 # Check h1 network configuration
2 mininet> h1 ifconfig
3 # Output
4 h1-eth0    Link encap:Ethernet  HWaddr a6:78:8c:84:c9:6b
5            inet addr:10.0.0.1  Bcast:10.255.255.255  Mask:255.0.0.0
6            UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
7            RX packets:0 errors:0 dropped:0 overruns:0 frame:0
8            TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
9            collisions:0 txqueuelen:1000
10           RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
11
12 lo        Link encap:Local Loopback
13           inet addr:127.0.0.1  Mask:255.0.0.0
14           UP LOOPBACK RUNNING  MTU:65536  Metric:1
15           RX packets:0 errors:0 dropped:0 overruns:0 frame:0
16           TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
17           collisions:0 txqueuelen:1
18           RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
```

Chaque hôte Mininet est un espace de noms réseau Linux indépendant (network namespace) avec sa propre interface virtuelle. h1 reçoit l'adresse IP 10.0.0.1. L'adresse MAC est générée aléatoirement par Mininet à chaque démarrage.

Test de connectivité ICMP entre h1 et h2 :

```
1 # Test ICMP connectivity
2 h1 ping h2 -c 4
3 # Output
4 PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
5 64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=41.3 ms
6 64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.070 ms
7 64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.062 ms
8 64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.051 ms
9 --- 10.0.0.2 ping statistics ---
10 4 packets transmitted, 4 received, 0% packet loss, time 2998ms
11 rtt min/avg/max/mdev = 0.051/10.372/41.307/17.860 ms
```

le résultat 0% packet loss confirme que h1 et h2 communiquent correctement.

Test de connectivité globale (pingall) :

```
1 # Test overall connectivity
2 pingall
3 # Output
4 *** Ping: testing ping reachability
5 h1 -> h2 h3
6 h2 -> h1 h3
7 h3 -> h1 h2
8 *** Results: 0% dropped (6/6 received)
```

Le résultat 0% dropped (6/6 received) confirme que tous les hôtes communiquent correctement et que le contrôleur POX gère le réseau dans son intégralité.

Affichage de la table de flux de s1 :

```
1 # Check s1 flow table
2 dpctl dump-flows
3 # Output
4 *** s1 -----
5 NXST_FLOW reply (xid=0x4):
6   cookie=0x0, duration=77.490s, table=0, n_packets=7, n_bytes=630, idle_age=47,
6   dl_src=a6:78:8c:84:c9:6b,dl_dst=86:16:5d:0c:3b:5f actions=output:2
7   cookie=0x0, duration=77.452s, table=0, n_packets=8, n_bytes=672, idle_age=47,
7   dl_src=86:16:5d:0c:3b:5f,dl_dst=a6:78:8c:84:c9:6b actions=output:1
8   cookie=0x0, duration=47.486s, table=0, n_packets=3, n_bytes=238, idle_age=42,
8   dl_src=a6:78:8c:84:c9:6b,dl_dst=1a:3b:3c:40:71:5a actions=output:3
9   cookie=0x0, duration=47.448s, table=0, n_packets=4, n_bytes=280, idle_age=42,
9   dl_src=1a:3b:3c:40:71:5a,dl_dst=a6:78:8c:84:c9:6b actions=output:1
10  cookie=0x0, duration=47.444s, table=0, n_packets=3, n_bytes=238, idle_age=42,
10  dl_src=86:16:5d:0c:3b:5f,dl_dst=1a:3b:3c:40:71:5a actions=output:3
11  cookie=0x0, duration=47.408s, table=0, n_packets=4, n_bytes=280, idle_age=42,
11  dl_src=1a:3b:3c:40:71:5a,dl_dst=86:16:5d:0c:3b:5f actions=output:2
```

Après le pingall, la table de flux de s1 contient 6 entrées : deux règles (une par sens) pour chacune des 3 paires d'hôtes possibles (h1↔h2, h1↔h3, h2↔h3).

Capture du trafic réseau avec Wireshark

3. Please log all packet exchanges using Wireshark.

L'objectif de cette étape est de capturer le trafic réseau généré lors de l'exécution d'un *simple web server* sur h1 et d'une requête HTTP depuis h2. L'infrastructure simulée pour ce test est la topologie single,3 avec le contrôleur POX en mode forwarding.l2_pairs.

Capturer le trafic réseau dans une première session SSH :

```
1 # Test overall connectivity
2 sudo tshark -i any -w /tmp/capture.pcap
```

Démarrer le serveur HTTP sur h1 et effectuer une requête depuis h2 dans une seconde session SSH :

```
1 h1 python -m SimpleHTTPServer 80 &
2 h2 wget -O - h1
```

Arrêter la capture (Ctrl+C) et modifier les permissions du fichier de capture pour permettre à l'utilisateur hôte de le lire (par défaut, il est créé avec des permissions restrictives) :

```
1 sudo chmod 644 /tmp/capture.pcap
```

Transférer le fichier de capture vers la machine hôte pour analyse avec Wireshark (commande à exécuter sur la machine hôte) :

```
1 scp mininet@192.168.56.101:/tmp/capture.pcap ~/capture.pcap
```

La capture peut ensuite être ouverte dans Wireshark sur la machine hôte pour analyse.

Échange d'initialisation OpenFlow

4. Isolate the OpenFlow initialization exchange (get inspiration from the course slides).

Pour isoler l'initialisation *OpenFlow*, filtrer sur `openflow_v1` dans *Wireshark*. Les paquets capturés sont présentés par Figure 1.

openflow_v1					
No.	Time	Source	Destination	Protocol	Info
102	1.371354450	127.0.0.1	127.0.0.1	OpenFlow	Type: OFPT_HELLO
104	1.371595634	127.0.0.1	127.0.0.1	OpenFlow	Type: OFPT_STATS_REQUEST
108	1.374203555	127.0.0.1	127.0.0.1	OpenFlow	Type: OFPT_FEATURES_REPLY
109	1.374215087	127.0.0.1	127.0.0.1	OpenFlow	Type: OFPT_STATS_REPLY
111	1.374557502	127.0.0.1	127.0.0.1	OpenFlow	Type: OFPT_SET_CONFIG
117	1.413612636	127.0.0.1	127.0.0.1	OpenFlow	Type: OFPT_BARRIER_REQUEST
119	1.413727523	127.0.0.1	127.0.0.1	OpenFlow	Type: OFPT_BARRIER_REPLY
155	6.270615197	127.0.0.1	127.0.0.1	OpenFlow	Type: OFPT_ECHO_REQUEST
157	6.270883252	127.0.0.1	127.0.0.1	OpenFlow	Type: OFPT_ECHO_REPLY
179	11.270741544	127.0.0.1	127.0.0.1	OpenFlow	Type: OFPT_ECHO_REQUEST
180	11.271020770	127.0.0.1	127.0.0.1	OpenFlow	Type: OFPT_ECHO_REPLY
202	16.270479450	127.0.0.1	127.0.0.1	OpenFlow	Type: OFPT_ECHO_REQUEST
203	16.270791237	127.0.0.1	127.0.0.1	OpenFlow	Type: OFPT_ECHO_REPLY
233	21.270494739	127.0.0.1	127.0.0.1	OpenFlow	Type: OFPT_ECHO_REQUEST
234	21.270804091	127.0.0.1	127.0.0.1	OpenFlow	Type: OFPT_ECHO_REPLY
258	23.992976148	8a:5d:a7:3b:49:43	Broadcast	OpenFlow	Type: OFPT_PACKET_IN
261	23.993666188	8a:5d:a7:3b:49:43	Broadcast	OpenFlow	Type: OFPT_PACKET_OUT
268	23.994487344	2a:b2:0a:d9:81:af	8a:5d:a7:3b:49:43	OpenFlow	Type: OFPT_PACKET_IN
269	23.994970143	127.0.0.1	127.0.0.1	OpenFlow	Type: OFPT_FLOW_MOD
279	24.033611799	2a:b2:0a:d9:81:af	8a:5d:a7:3b:49:43	OpenFlow	Type: OFPT_PACKET_OUT
281	24.033664268	127.0.0.1	127.0.0.1	OpenFlow	Type: OFPT_BARRIER_REPLY

Figure 1: Paquets OpenFlow capturés lors de l'initialisation entre le contrôleur et le switch.

Une première séquence d'échange de 14 paquets se fait entre le *controller* (127.0.0.1) et le *switch* (127.0.0.1).

- Le premier paquet (No. 102) est de type `OFPT_HELLO`, il est utilisé pour établir la connexion et négocier la version du protocole à utiliser (dans notre cas, version 1.0).
- Le deuxième paquet (No. 104) est de type `OFPT_STATS_REQUESTS`, il permet de récupérer différentes informations sur le *switch*. Le paquet No. 109 est la réponse du *switch* à ce paquet.
- Le troisième paquet (No. 108) est de `OFPT_STATS_REPLY`, c'est une réponse du *switch* à une requête d'information sur ses capacités et ses ports.
- Le paquet No. 109 est de type `OFPT_SET_CONFIG` et permet de configurer le *switch*.
- Le paquet No. 117 est de type `OFPT_BARRIER_REQUEST` permet de synchroniser le *controller* et le *switch* et d'ordonner ainsi les messages. Le paquet No. 119 est la réponse du *switch* à ce paquet.
- Les paquets No. 155, 179, 202, 233 sont de type `OFPT_ECHO_REQUEST`, envoyé par le *controller* pour vérifier que la connexion avec le *switch* est toujours active et mesurer la latence. Les paquets No. 119, 157, 180, 203, 234 sont les `OFPT_ECHO_REPLY` du *switch* confirmant la réception.

Puis une séquence d'échange de 4 paquets entre h2, h1, le *switch* et le contrôleur, liés à la résolution ARP transitant par le plan de contrôle OpenFlow :

- Le paquet No. 258 de type `OFPT_PACKET_IN` est envoyé par le *switch* au contrôleur lorsqu'il reçoit une requête ARP de h2 (10.0.0.2) vers l'adresse *broadcast*, sans règle de flux correspondante. Le contrôleur répond avec un `OFPT_PACKET_OUT` pour indiquer au *switch* comment transmettre ce paquet. Une deuxième interaction similaire a lieu avec les paquets No. 268 et No. 279, cette fois pour la réponse ARP de h1 (10.0.0.1) vers h2.

Finalement, une fois les adresses MAC de h1 et h2 apprises, le contrôleur installe une règle de flux dans le *switch* :

- Le paquet No. 269 de type `OFPT_FLOW_MOD` est envoyé par le contrôleur pour installer, modifier ou supprimer une règle de flux dans la table du *switch*, afin que les prochains paquets similaires soient traités directement sans repasser par le contrôleur.

Requêtes et réponses ARP

5. Try identifying the ARP requests and replies (get inspired from the course slides).

Pour isoler les requêtes ARP, filtrer sur `arp` dans *Wireshark*. Les paquets capturés sont présentés par Figure 2.

arp					
No.	Time	Source	Destination	Protocol	Info
255	23.991524044	8a:5d:a7:3b:49:43		ARP	Who has 10.0.0.1? Tell 10.0.0.2
258	23.992976148	8a:5d:a7:3b:49:43	Broadcast	OpenFlow	Type: OFPT_PACKET_IN
261	23.993666188	8a:5d:a7:3b:49:43	Broadcast	OpenFlow	Type: OFPT_PACKET_OUT
265	23.994294301	8a:5d:a7:3b:49:43		ARP	Who has 10.0.0.1? Tell 10.0.0.2
266	23.994296885	8a:5d:a7:3b:49:43		ARP	Who has 10.0.0.1? Tell 10.0.0.2
267	23.994315010	2a:b2:0a:d9:81:af		ARP	10.0.0.1 is at 2a:b2:0a:d9:81:af
268	23.994487344	2a:b2:0a:d9:81:af	8a:5d:a7:3b:49:43	OpenFlow	Type: OFPT_PACKET_IN
279	24.033611799	2a:b2:0a:d9:81:af	8a:5d:a7:3b:49:43	OpenFlow	Type: OFPT_PACKET_OUT
282	24.033802519	2a:b2:0a:d9:81:af		ARP	10.0.0.1 is at 2a:b2:0a:d9:81:af

Figure 2: Paquets ARP capturés lors de la résolution d'adresse entre h1 et h2.

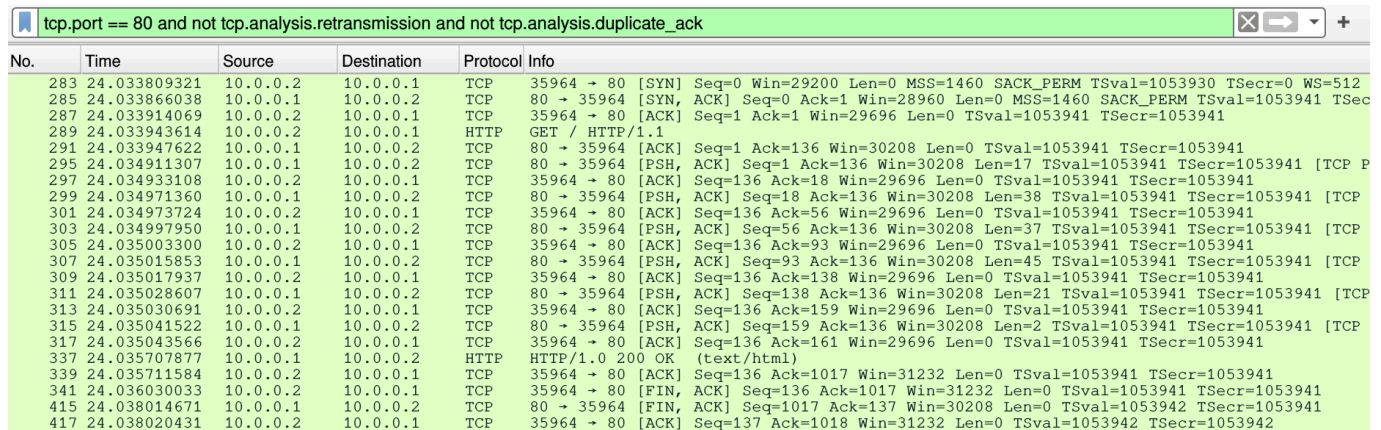
- Les paquets No. 255, No. 265 et No. 266 proviennent de h2 (8a:5d:a7:3b:49:43) et sont des requêtes ARP (*ARP request*) demandant Who has 10.0.0.1? Tell 10.0.0.2, diffusées en *broadcast* pour résoudre l'adresse MAC de h1.
- Les paquets No. 267 et No. 282 proviennent de h1 (2a:b2:0a:d9:81:af) et ont le contenu 10.0.0.1 is at 2a:b2:0a:d9:81:af. Ce sont des réponses ARP (*ARP reply*) aux paquets No. 255, No. 265 et No. 266.

Les échanges `OFPT_PACKET_IN` et `OFPT_PACKET_OUT` associés à ces paquets ARP sont décrits dans la section précédente.

Création de socket ICMP et HTTP

6. Isolate and comment the socket creation exchanges using both ICMP and HTTP.

Pour isoler le trafic HTTP, filtrer sur `tcp.port == 80` dans *Wireshark* (voir Figure 3).



No.	Time	Source	Destination	Protocol	Info
283	24.033809321	10.0.0.2	10.0.0.1	TCP	35964 → 80 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK_PERM TSval=1053930 TSecr=0 WS=512
285	24.033866038	10.0.0.1	10.0.0.2	TCP	80 → 35964 [SYN, ACK] Seq=0 Ack=1 Win=28960 Len=0 MSS=1460 SACK_PERM TSval=1053941 TSecr=0
287	24.033914069	10.0.0.2	10.0.0.1	TCP	35964 → 80 [ACK] Seq=1 Ack=1 Win=29696 Len=0 TSval=1053941 TSecr=1053941
289	24.033943614	10.0.0.2	10.0.0.1	HTTP	GET / HTTP/1.1
291	24.033947622	10.0.0.1	10.0.0.2	TCP	80 → 35964 [ACK] Seq=1 Ack=136 Win=30208 Len=0 TSval=1053941 TSecr=1053941
295	24.034911307	10.0.0.1	10.0.0.2	TCP	80 → 35964 [PSH, ACK] Seq=1 Ack=136 Win=30208 Len=17 TSval=1053941 TSecr=1053941 [TCP P
297	24.034933108	10.0.0.2	10.0.0.1	TCP	35964 → 80 [ACK] Seq=136 Ack=18 Win=29696 Len=0 TSval=1053941 TSecr=1053941
299	24.034971360	10.0.0.1	10.0.0.2	TCP	80 → 35964 [PSH, ACK] Seq=18 Ack=136 Win=30208 Len=38 TSval=1053941 TSecr=1053941 [TCP
301	24.034973724	10.0.0.2	10.0.0.1	TCP	35964 → 80 [ACK] Seq=136 Ack=56 Win=29696 Len=0 TSval=1053941 TSecr=1053941
303	24.034997950	10.0.0.1	10.0.0.2	TCP	80 → 35964 [PSH, ACK] Seq=56 Ack=136 Win=30208 Len=37 TSval=1053941 TSecr=1053941 [TCP
305	24.035003300	10.0.0.2	10.0.0.1	TCP	35964 → 80 [ACK] Seq=136 Ack=93 Win=29696 Len=0 TSval=1053941 TSecr=1053941
307	24.035015853	10.0.0.1	10.0.0.2	TCP	80 → 35964 [PSH, ACK] Seq=93 Ack=136 Win=30208 Len=45 TSval=1053941 TSecr=1053941 [TCP
309	24.035017937	10.0.0.2	10.0.0.1	TCP	35964 → 80 [ACK] Seq=136 Ack=138 Win=29696 Len=0 TSval=1053941 TSecr=1053941
311	24.035028607	10.0.0.1	10.0.0.2	TCP	80 → 35964 [PSH, ACK] Seq=138 Ack=136 Win=30208 Len=21 TSval=1053941 TSecr=1053941 [TCP
313	24.035030691	10.0.0.2	10.0.0.1	TCP	35964 → 80 [ACK] Seq=136 Ack=159 Win=29696 Len=0 TSval=1053941 TSecr=1053941
315	24.035041522	10.0.0.1	10.0.0.2	TCP	80 → 35964 [PSH, ACK] Seq=159 Ack=136 Win=30208 Len=2 TSval=1053941 TSecr=1053941 [TCP
317	24.035043566	10.0.0.2	10.0.0.1	TCP	35964 → 80 [ACK] Seq=136 Ack=161 Win=29696 Len=0 TSval=1053941 TSecr=1053941
337	24.035707877	10.0.0.1	10.0.0.2	HTTP	HTTP/1.0 200 OK (text/html)
339	24.035711584	10.0.0.2	10.0.0.1	TCP	35964 → 80 [ACK] Seq=136 Ack=1017 Win=31232 Len=0 TSval=1053941 TSecr=1053941
341	24.036030033	10.0.0.2	10.0.0.1	TCP	35964 → 80 [FIN, ACK] Seq=136 Ack=1017 Win=31232 Len=0 TSval=1053941 TSecr=1053941
415	24.038014671	10.0.0.1	10.0.0.2	TCP	80 → 35964 [FIN, ACK] Seq=1017 Ack=137 Win=30208 Len=0 TSval=1053942 TSecr=1053941
417	24.038020431	10.0.0.2	10.0.0.1	TCP	35964 → 80 [ACK] Seq=137 Ack=1018 Win=31232 Len=0 TSval=1053942 TSecr=1053942

Figure 3: Échanges TCP/HTTP entre h2 et h1 : 3-way *handshake*, requête GET et réponse 200 OK, puis fermeture de connexion.

La création de la socket TCP entre h2 (10.0.0.2) et h1 (10.0.0.1) suit le 3-way *handshake* standard :

- Le paquet No. 283 est un SYN envoyé par h2 vers h1 (port 35964 → 80), initiant la connexion.
- Le paquet No. 285 est le SYN-ACK de h1 en réponse, acceptant la connexion.
- Le paquet No. 287 est le ACK final de h2, confirmant l'établissement de la socket.

Une fois la socket établie, l'échange applicatif peut avoir lieu :

- Le paquet No. 289 est la requête GET / HTTP/1.1 envoyée par h2 vers h1.
- Les paquets No. 295 à No. 315 sont des segments TCP (PSH, ACK) transportant des fragments de la réponse HTTP. La réponse étant trop grande pour tenir dans un seul paquet, TCP la découpe en plusieurs segments. *Wireshark* les affiche comme TCP (et non HTTP) car chaque segment est incomplet.
- Le paquet No. 337 est le dernier segment : *Wireshark* réassemble l'ensemble et affiche HTTP/1.0 200 OK — c'est à ce moment qu'il reconnaît le message HTTP complet.

Finalement, la socket est fermée via un 4-way *handshake* :

- Le paquet No. 341 est un FIN, ACK envoyé par h2 vers h1, initiant la fermeture de la connexion.
- Le paquet No. 415 est le FIN, ACK de h1 vers h2, confirmant la fermeture de son côté.
- Le paquet No. 417 est le ACK final de h2, clôturant définitivement la socket.

Pour isoler le trafic ICMP, filtrer sur `icmp` dans *Wireshark* (voir Figure 4).

icmp and not icmp.type == 3									
No.	Time	Source	Destination	Protocol	Info				
920	89.596764324	10.0.0.2	10.0.0.1	ICMP	Echo (ping) request	id=0x04d2, seq=1/256, ttl=64	(no response found!)		
921	89.596799069	10.0.0.2	10.0.0.1	ICMP	Echo (ping) request	id=0x04d2, seq=1/256, ttl=64	(reply in 922)		
922	89.596810591	10.0.0.1	10.0.0.2	ICMP	Echo (ping) reply	id=0x04d2, seq=1/256, ttl=64	(request in 921)		
923	89.596840026	10.0.0.1	10.0.0.2	ICMP	Echo (ping) reply	id=0x04d2, seq=1/256, ttl=64			
933	90.555762177	10.0.0.2	10.0.0.1	ICMP	Echo (ping) request	id=0x04d2, seq=2/512, ttl=64	(no response found!)		
934	90.555771134	10.0.0.2	10.0.0.1	ICMP	Echo (ping) request	id=0x04d2, seq=2/512, ttl=64	(reply in 935)		
935	90.555786803	10.0.0.1	10.0.0.2	ICMP	Echo (ping) reply	id=0x04d2, seq=2/512, ttl=64	(request in 934)		
936	90.555788877	10.0.0.1	10.0.0.2	ICMP	Echo (ping) reply	id=0x04d2, seq=2/512, ttl=64			
945	91.554758768	10.0.0.2	10.0.0.1	ICMP	Echo (ping) request	id=0x04d2, seq=3/768, ttl=64	(no response found!)		
946	91.554767524	10.0.0.2	10.0.0.1	ICMP	Echo (ping) request	id=0x04d2, seq=3/768, ttl=64	(reply in 947)		
947	91.554779276	10.0.0.1	10.0.0.2	ICMP	Echo (ping) reply	id=0x04d2, seq=3/768, ttl=64	(request in 946)		
948	91.554781400	10.0.0.1	10.0.0.2	ICMP	Echo (ping) reply	id=0x04d2, seq=3/768, ttl=64			

Figure 4: Paquets ICMP *Echo request* et *Echo reply* entre h2 (10.0.0.2) et h1 (10.0.0.1) lors du ping.

Les paquets ICMP observés sont :

- Les paquets No. 920 et No. 922 correspondent au premier *ping* (seq=1) : *Echo request* de h2 (10.0.0.2) vers h1 (10.0.0.1) et *Echo reply* en retour.
- Les paquets No. 933 et No. 935 correspondent au deuxième *ping* (seq=2).
- Les paquets No. 945 et No. 947 correspondent au troisième *ping* (seq=3).

Contrairement au protocole TCP, ICMP est sans connexion (*connectionless*) : il n'y a ni établissement ni fermeture de socket. Chaque *Echo request* est une communication indépendante.

Conclusion

Ce laboratoire a permis de mettre en pratique les principes du *Software-Defined Networking* à travers un environnement *Mininet* complet. L'analyse des captures *Wireshark* a confirmé le comportement attendu du protocole *OpenFlow* : initialisation via l'échange HELLO / FEATURES_REPLY / SET_CONFIG, apprentissage des adresses MAC par le contrôleur *POX* via les messages OFPT_PACKET_IN, installation des règles de flux avec OFPT_FLOW_MOD, et transmission directe des paquets suivants par le *switch* sans intervention du contrôleur.

Les échanges ARP, ICMP et HTTP ont illustré concrètement la séparation entre plan de contrôle et plan de données : les premiers paquets d'un flux inconnu transitent par le contrôleur, tandis que les suivants sont forwardés directement par le *switch* grâce aux règles installées. Cette différence est particulièrement visible sur le trafic HTTP, où le 3-way *handshake* TCP matérialise la création de la *socket*, et sur ICMP, où tous les pings passent directement par le *switch* dès lors que la règle de flux est en place.